



Automated Bridge Surface Crack Detection

Using UAV Imagery & Deep Learning Segmentation

Motivation & Problem Statement

Safety Risk

Inspectors exposed to traffic, heights, and hazardous environments during manual evaluations.

Time Constraints

Traditional inspections are slow and expensive, delaying critical maintenance decisions.

Human Error

Visual assessments vary across inspectors, leading to inconsistent crack identification.

Problem Statement

Can we use high resolution UAV RGB imagery to automatically detect and localize surface cracks on bridge infrastructure, reducing the need for manual, risky inspections?

Goal: Produce pixel-level segmentation masks that identify cracked regions on bridge surfaces, enabling engineers to efficiently prioritize repairs.

Data Description

Primary Dataset

SDNET2018

Concrete crack dataset for ML applications. Contains labeled images of cracked and non-cracked concrete surfaces from bridges and pavements.

Primary Dataset

UAV-Based Crack Detection (Kaggle)

UAV captured imagery with ground-truth segmentation masks. Includes images from 30 bridges using drones.



INPUTS

- High resolution RGB drone images of bridge surfaces
- Tiled to 256×256 or 512×512 pixel patches
- Normalized pixel values (0–1 float)
- Augmented: flips, rotations, brightness shifts



OUTPUTS

- Binary pixel-level probability mask (crack / no crack)
- Same spatial resolution as input tile
- Per-pixel class confidence in [0, 1]
- Threshold at 0.5 for final binary prediction

Data Split: Training 70% · Validation 15% · Test 15% | Source: 30 bridges

Methodology

Proposed Architecture: U-Net CNN

► Encoder (Contracting Path)

Repeated 3×3 conv + ReLU + max pooling. Extracts hierarchical features; progressively reduces spatial dims.

► Bottleneck

Deepest feature representation capturing high-level semantic crack patterns.

► Decoder (Expanding Path)

Transposed convolutions upsample back to original resolution. Skip connections restore spatial detail.

► Skip Connections

Forward fine-grained edge & texture info from encoder to decoder, critical for thin crack localization.

Loss Function

Combined BCE + Dice Loss

Binary Cross-Entropy handles pixel-wise classification.

Dice Loss directly optimizes segmentation overlap, minimizing the impact of heavy class imbalance since crack pixels are a tiny fraction of total image area.

Evaluation Metrics

Recall

PRIMARY — target ≥ 0.90 . Minimize missed cracks.

Dice / F1

PRIMARY — segmentation overlap quality.

Mean IoU

Secondary — intersection over union.

Precision

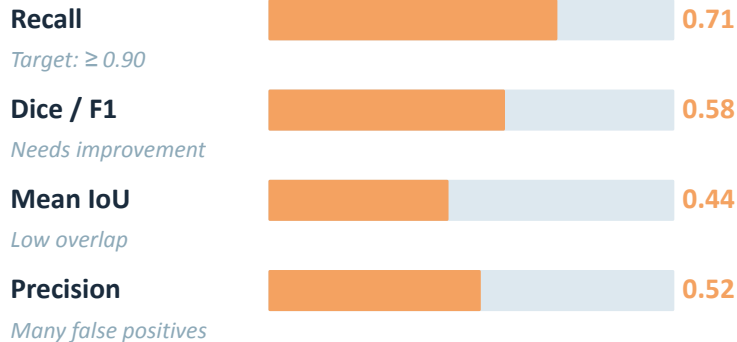
Secondary — false positive monitor.

Baseline Model & Performance

Baseline: Random Forest Classifier

- Hand-crafted patch-level features: texture (LBP), gradient (HOG), intensity histograms
- Each 16×16 sub-patch classified independently as crack or no-crack
- 100 decision trees, max depth 20, `class_weight='balanced'`
- No spatial context — treats each patch in isolation

Baseline Results (Test Set)



Analysis

The Random Forest baseline achieves moderate recall (0.71) but falls well short of the 0.90 target. Patch-level feature engineering misses fine spatial context — cracks spanning patch boundaries are fragmented. High class imbalance leads to many false negatives despite `class_weight='balanced'`. This establishes a clear lower bound; the U-Net architecture is expected to significantly improve all metrics through end-to-end pixel-wise learning and skip connections.

Prior Work & Next Steps

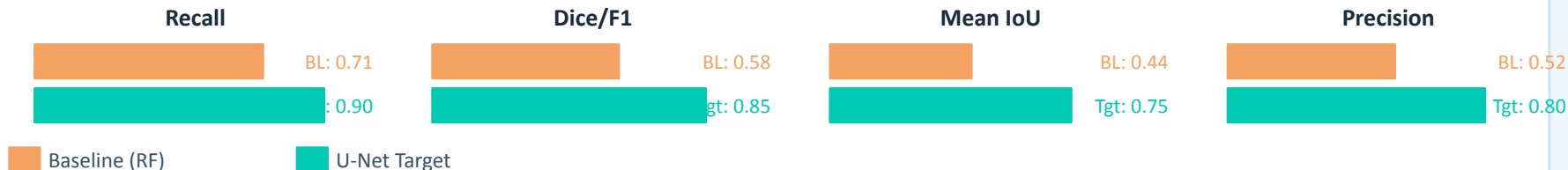
Prior Work Completed

- Surveyed open-source bridge/crack datasets (SDNET2018, Kaggle UAV dataset)
- Reviewed U-Net and related encoder-decoder architectures for pixel segmentation
- Studied class imbalance mitigation strategies: Dice Loss, focal loss, oversampling
- Implemented and evaluated Random Forest baseline classifier
- Defined primary and secondary evaluation metric targets

Next Steps

- Implement full U-Net architecture in PyTorch/TensorFlow
- Train with combined BCE + Dice Loss on 70/15/15 split
- Apply data augmentation (flips, rotations, color jitter)
- Tune threshold for Recall ≥ 0.90 target
- Compare U-Net performance against RF baseline

Target vs. Baseline Comparison



References

- [1] S. Dorafshan, R. J. Thomas, and M. Maguire, "SDNET2018: A concrete crack image dataset for machine learning applications," Utah State University, 2018. https://digitalcommons.usu.edu/all_datasets/48/
- [2] UAV-Based Crack Detection Dataset, Kaggle. <https://www.kaggle.com/datasets/canaria/uav-based-crack-detection>
- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015. arXiv:1505.04597
- [4] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," 3DV, 2016. (Dice Loss formulation)
- [5] S. Dorafshan, R. J. Thomas, and M. Maguire, "Comparison of Deep Convolutional Neural Networks and Edge Detectors for Image-Based Crack Detection in Concrete," Construction and Building Materials, 2018.